INF712 - Computerunterstützte Statistik

26-27 WS

26/10/2026

Inhaltsverzeichnis

1 Vorlesung 03: Monte-Carlo-Simulationen mit Stichproben **1**

1.1 Zufall und Pseudozufall 2
1.2 Erste Algorithmen 5
1.3 Zufallszahlengeneratoren testen 11
1.4 Die Monte-Carlo-Methode 13
1.5 Monte-Carlo-Schätzung von π 14
1.6 Monte-Carlo-Simulation: Kollisionen in Hashtabellen 23
1.7 Ergänzungen 32

1 Vorlesung 03: Monte-Carlo-Simulationen mit Stichproben

Zufallszahlen erzeugen, ihre Qualität prüfen und mit ihnen rechnen: Pseudozufall, erste Generatoren, Schätzung von π, Kollisionen in Hashtabellen.

Lernziele

Nach dieser Vorlesung können Sie

- echten Zufall und Pseudozufall unterscheiden und erklären, warum die Statistik bewusst mit Pseudozufall arbeitet,
- die Mittel-Quadrat-Methode und lineare Kongruenzgeneratoren in R implementieren und ihre Schwächen nachweisen,
- einen Zufallszahlengenerator mit Histogramm, χ^2-, KS-, Runs- und seriellem Test prüfen und die Ergebnisse richtig deuten,
- mit der Inversionsmethode exponential- und normalverteilte Zufallszahlen aus gleichverteilten erzeugen,

- eine Größe per Monte-Carlo-Simulation schätzen und den Simulationsfehler mit einem Standardfehler angeben,
- Kollisionswahrscheinlichkeiten in Hashtabellen exakt, approximativ und durch Simulation bestimmen.

1.1 Zufall und Pseudozufall

Was heißt „zufällig"?

Eine Folge ist für uns *zufällig*, wenn man ihre nächsten Werte nicht vorhersagen kann — auch nicht mit Kenntnis aller bisherigen Werte. Für statistische Zwecke verlangen wir zusätzlich bestimmte Verteilungseigenschaften: Die Werte sollen etwa gleichverteilt und voneinander unabhängig sein.

Dabei muss man drei Arten von „Zufall" unterscheiden:

Art	Quelle	Vorhersagbar?	Reproduzierbar?
Echter Zufall	Quantenprozesse: radioaktiver Zerfall, Photon am Strahlteiler, Vakuumfluktuationen	prinzipiell nein	nein
Physikalischer (chaotischer) Zufall	Würfel, Münze, Roulette, Lottotrommel, thermisches Rauschen	prinzipiell ja, praktisch nein	nein
Pseudozufall	deterministischer Algorithmus mit Startwert (*seed*)	ja, bei bekanntem Zustand	ja

Echter Zufall: Quantenmechanik

Nach heutigem Verständnis der Physik ist der Ausgang eines einzelnen Quantenereignisses nicht nur *unbekannt*, sondern *unbestimmt*:

- Ein Photon trifft auf einen halbdurchlässigen Spiegel und wird mit Wahrscheinlichkeit 1/2 durchgelassen oder reflektiert. Ein Detektor hinter jedem Ausgang liefert ein Bit.
- Ein radioaktiver Kern zerfällt zu einem nicht vorhersagbaren Zeitpunkt; die Wartezeiten sind exponentialverteilt.

Solche *True Random Number Generators* (TRNG) gibt es als Hardware (Quanten-Zufallsgeneratoren, Zufallsgeneratoren in Prozessoren auf Basis thermischen Rauschens) und

als öffentliche Dienste. Sie sind langsam, teuer und — für die Statistik entscheidend — **nicht wiederholbar**.

Physikalischer Zufall: der Würfel

Ein Würfelwurf folgt den Gesetzen der klassischen Mechanik. Wären Anfangslage, Geschwindigkeit, Drehimpuls, Tischoberfläche und Luftwiderstand exakt bekannt, ließe sich das Ergebnis berechnen. Der Wurf ist also **deterministisch**, aber **chaotisch**: Winzige Änderungen der Anfangsbedingungen ändern das Ergebnis vollständig. Weil wir die Anfangsbedingungen nie genau genug kennen, *verhält* sich der Würfel zufällig.

In diesem Sinn ist der Würfel dem Pseudozufallsgenerator näher als dem Quantenzufall: ein deterministisches System, dessen Zustand wir nicht kennen. Dass die Grenze unscharf ist, zeigt der Münzwurf: Persi Diaconis und Kollegen haben gezeigt, dass eine von Hand geworfene Münze mit etwa 51 % auf derselben Seite landet, mit der sie gestartet ist — die „Zufälligkeit" ist nicht perfekt.

> **Zufall aus der Physik in der Informatik**
>
> Betriebssysteme sammeln *Entropie* aus schwer vorhersagbaren Ereignissen (Zeitpunkte von Interrupts, Tastenanschläge, Netzwerkpakete) und speisen damit `/dev/random` bzw. `/dev/urandom`. Ein bekanntes Beispiel für physikalischen Zufall ist eine Wand aus Lavalampen, deren Kamerabilder ein Internetunternehmen als zusätzliche Entropiequelle für kryptografische Schlüssel nutzt.

Pseudozufall

Ein *Pseudozufallszahlengenerator* (PRNG) ist eine deterministische Funktion

$$s_{n+1} = f(s_n), \qquad u_n = g(s_n) \in [0, 1),$$

die aus einem internen **Zustand** s_n den nächsten Zustand und eine Ausgabe u_n berechnet. Der Startzustand s_0 ist der **seed**.

Daraus folgen drei Eigenschaften, die man kennen muss:

1. **Gleicher seed, gleiche Folge.** Das ist kein Mangel, sondern genau das, was wir in der Statistik wollen: Reproduzierbarkeit (`set.seed()`).
2. **Endliche Periode.** Der Zustandsraum ist endlich, also wiederholt sich die Folge irgendwann. Gute Generatoren haben eine Periode, die in keiner Anwendung erreicht wird.
3. **Kein echter Zufall.** Wer den Zustand kennt, kennt die Zukunft.

John von Neumann, selbst Erfinder eines der ersten Generatoren, warnte sinngemäß, wer Zufallsziffern mit arithmetischen Methoden erzeuge, befinde sich „in a state of sin". Gemeint ist: Man darf Pseudozufallszahlen nicht für echten Zufall *halten* — man darf sie aber *verwenden*, solange sie die nötigen Tests bestehen.

Anforderungen an einen guten PRNG

Anforderung	Bedeutung
Gleichverteilung	Alle Teilintervalle von $[0, 1)$ werden gemäß ihrer Länge getroffen
Unabhängigkeit	Aufeinanderfolgende Werte zeigen keine erkennbare Abhängigkeit, auch nicht in Paaren, Tripeln, …
Lange Periode	Die Folge wiederholt sich nicht innerhalb einer Simulation
Reproduzierbarkeit	Gleicher seed liefert dieselbe Folge — auf jedem Rechner
Geschwindigkeit	Millionen Zahlen pro Sekunde
(Kryptografische Sicherheit)	Aus bisherigen Ausgaben ist die nächste nicht berechenbar — nur für Sicherheitsanwendungen nötig

TRNG, PRNG, CSPRNG

	TRNG	PRNG (Statistik)	CSPRNG (Kryptografie)
Beispiel	Quantengenerator	Mersenne-Twister	ChaCha20, `/dev/urandom`
Reproduzierbar	nein	ja	nur mit Schlüssel
Schnell	nein	sehr	schnell
Vorhersagbar	nein	ja (Mersenne-Twister: nach 624 Ausgaben ist der Zustand rekonstruierbar)	praktisch nein
Einsatz	Seeds, Schlüssel	Simulation, Statistik	Passwörter, Tokens, Schlüssel

> ⚠ Nie für Sicherheit
>
> `runif()`, `sample()` und `rnorm()` sind für Simulationen gebaut, nicht für Passwörter, Session-Tokens oder Schlüssel. Dafür gibt es kryptografisch sichere Generatoren (in R z. B. das Paket `openssl` mit `openssl::rand_bytes()`).

Der Generator von R

```
RNGkind()
#> [1] "Mersenne-Twister" "Inversion"        "Rejection"
```

Die Ausgabe nennt drei Generatoren: für gleichverteilte Zahlen (*Mersenne-Twister*, Matsumoto & Nishimura 1998, Periode $2^{19937} - 1$), für normalverteilte Zahlen (*Inversion*, also `qnorm(runif())`) und für `sample()` (*Rejection*).

```
set.seed(2026)
runif(5)
#> [1] 0.6986735 0.5565305 0.1401400 0.2857233 0.5553690
set.seed(2026)
runif(5)          # exakt dieselben Werte
#> [1] 0.6986735 0.5565305 0.1401400 0.2857233 0.5553690
```

> **`sample()` vor und nach R 3.6.0**
>
> Mit R 3.6.0 wurde das Verfahren von `sample()` geändert, weil das alte bei großen Populationen leicht ungleichmäßig war. Code aus älteren Lehrbüchern liefert daher trotz gleichem `set.seed()` andere Stichproben. Das alte Verhalten bekommt man mit `RNGkind(sample.kind = "Rounding")` — nur zum Nachrechnen alter Ergebnisse verwenden.

1.2 Erste Algorithmen

Die Mittel-Quadrat-Methode (von Neumann, 1946)

Idee: Man nimmt eine k-stellige Zahl, quadriert sie (ergibt bis zu $2k$ Stellen, links mit Nullen aufgefüllt) und nimmt die mittleren k Ziffern als nächste Zahl.

Beispiel mit $k = 4$: $1234^2 = 01522756$, die Mitte ist 5227. Weiter: $5227^2 = 27321529 \to 3215$, usw.

```
mittel_quadrat <- function(x, k = 4) {
  # Quadrat bilden, k/2 Stellen rechts abschneiden, k Stellen behalten
  floor(x^2 / 10^(k / 2)) %% 10^k
}

ms_folge <- function(seed, n, k = 4) {
  x <- numeric(n)
  x[1] <- seed
  for (i in 2:n) x[i] <- mittel_quadrat(x[i - 1], k)
```

```
  x
}

ms_folge(1234, 15)
#>  [1] 1234 5227 3215 3362 3030 1809 2724 4201 6484  422 1780 1684 8358 8561 2907
```

Das sieht zunächst zufällig aus. Andere Startwerte zeigen das Problem:

```
ms_folge(2100, 10)   # Zyklus der Länge 4
#>  [1] 2100 4100 8100 6100 2100 4100 8100 6100 2100 4100
ms_folge(3792, 5)    # Fixpunkt: 3792^2 = 14379264
#> [1] 3792 3792 3792 3792 3792
ms_folge(1009, 12)   # endet in 0 - und bleibt dort
#>  [1] 1009  180  324 1049 1004   80   64   40   16    2    0    0
```

Zyklen finden

Jede Folge eines deterministischen Generators mit endlichem Zustandsraum mündet in einen Zyklus. Die folgende Funktion funktioniert für *jede* Iterationsvorschrift `f` und liefert die Länge des Vorlaufs (bis der Zyklus erreicht ist) und die Zykluslänge:

```
zyklus <- function(f, seed) {
  gesehen <- seed
  x <- f(seed)
  while (!(x %in% gesehen)) {
    gesehen <- c(gesehen, x)
    x <- f(x)
  }
  pos <- match(x, gesehen)
  c(vorlauf = pos - 1, zyklus = length(gesehen) - pos + 1)
}

zyklus(mittel_quadrat, 1234)
#> vorlauf  zyklus
#>      56       1
zyklus(mittel_quadrat, 2100)
#> vorlauf  zyklus
#>       0       4
```

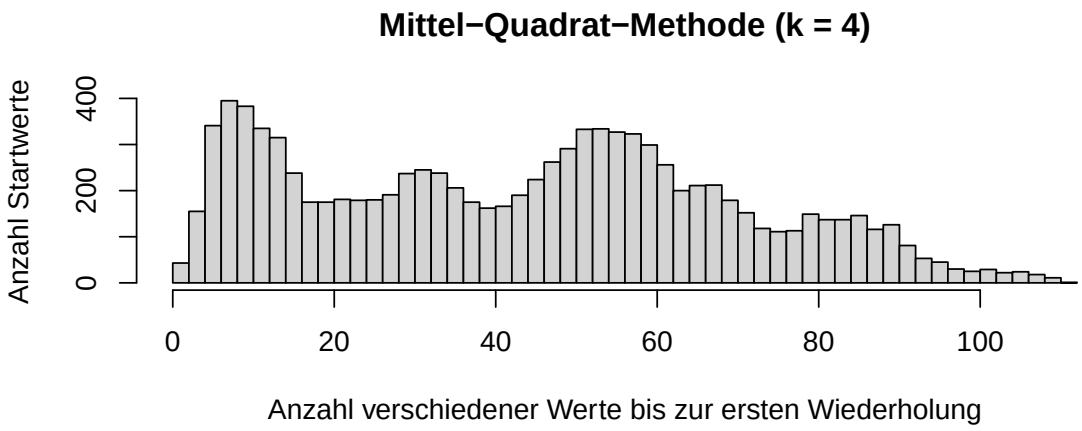
Jetzt untersuchen wir **alle** 10 000 vierstelligen Startwerte:

```r
erg <- t(sapply(0:9999, function(s) zyklus(mittel_quadrat, s)))
gesamt <- erg[, "vorlauf"] + erg[, "zyklus"]

table(erg[, "zyklus"])        # welche Zykluslängen kommen vor?
#>
#>    1    4
#> 2263 7737
max(gesamt)                   # längste Folge ohne Wiederholung
#> [1] 111
mean(gesamt)                  # mittlere Länge
#> [1] 43.7054
```

Es gibt nur zwei Zykluslängen (1 und 4), und keine Folge liefert mehr als 111 verschiedene Werte — im Mittel sind es rund 44, obwohl 10 000 Zustände möglich wären. Die Methode ist für Simulationen unbrauchbar. Sie ist trotzdem lehrreich: Sie zeigt, dass „kompliziert aussehende" Rechenvorschriften keineswegs gute Zufallszahlen liefern.

```r
hist(gesamt, breaks = 40, main = "Mittel-Quadrat-Methode (k = 4)",
     xlab = "Anzahl verschiedener Werte bis zur ersten Wiederholung",
     ylab = "Anzahl Startwerte")
```



Lineare Kongruenzgeneratoren (LCG, Lehmer 1949)

Der Klassiker unter den Generatoren:

$$x_{n+1} = (a \cdot x_n + c) \bmod m, \qquad u_n = \frac{x_n}{m}$$

mit Modul m, Multiplikator a, Inkrement c und seed x_0. Ist $c = 0$, spricht man von einem *multiplikativen* LCG.

```
lcg <- function(n, seed, a, c, m) {
  x <- numeric(n)
  zustand <- seed
  for (i in 1:n) {
    zustand <- (a * zustand + c) %% m
    x[i] <- zustand
  }
  x               # ganzzahlige Zustände; durch m teilen ergibt u in [0, 1)
}

lcg(20, seed = 7, a = 5, c = 3, m = 16)
#>  [1]  6  1  8 11 10  5 12 15 14  9  0  3  2 13  4  7  6  1  8 11
```

Mit $m = 16$ gibt es nur 16 mögliche Zustände; die Folge oben durchläuft alle 16, bevor sie sich wiederholt. Das ist kein Zufall, sondern folgt aus der Wahl der Parameter.

Satz von Hull und Dobell (1962). Ein LCG mit $c \neq 0$ hat genau dann die volle Periode m, wenn

1. c und m teilerfremd sind,
2. $a - 1$ durch jeden Primteiler von m teilbar ist,
3. $a - 1$ durch 4 teilbar ist, falls m durch 4 teilbar ist.

```
lcg_schritt <- function(a, c, m) function(x) (a * x + c) %% m

zyklus(lcg_schritt(a = 5, c = 3, m = 16), 7)   # alle Bedingungen erfüllt
#> vorlauf  zyklus
#>       0      16
zyklus(lcg_schritt(a = 4, c = 3, m = 16), 7)   # a - 1 = 3 ist ungerade
#> vorlauf  zyklus
#>       1       1
zyklus(lcg_schritt(a = 5, c = 4, m = 16), 7)   # c und m nicht teilerfremd
#> vorlauf  zyklus
#>       0       1
```

Bekannte Parameter

Name	a	c	m
„Minimal Standard" (Park & Miller)	16807	0	$2^{31} - 1$
RANDU (IBM, 1960er)	65539	0	2^{31}
ANSI-C `rand()` (Beispielimplementierung)	1103515245	12345	2^{31}

> **Genauigkeit in R**
>
> R rechnet mit Gleitkommazahlen (double), die ganze Zahlen nur bis 2^{53} exakt darstellen. Bei Park & Miller und RANDU bleibt $a \cdot x < 2^{53}$, die Rechnung ist exakt. Beim ANSI-C-Generator nicht — dort entstehen stille Rundungsfehler. Für eigene Experimente deshalb Parameter wählen, bei denen $a \cdot m < 2^{53}$ gilt.

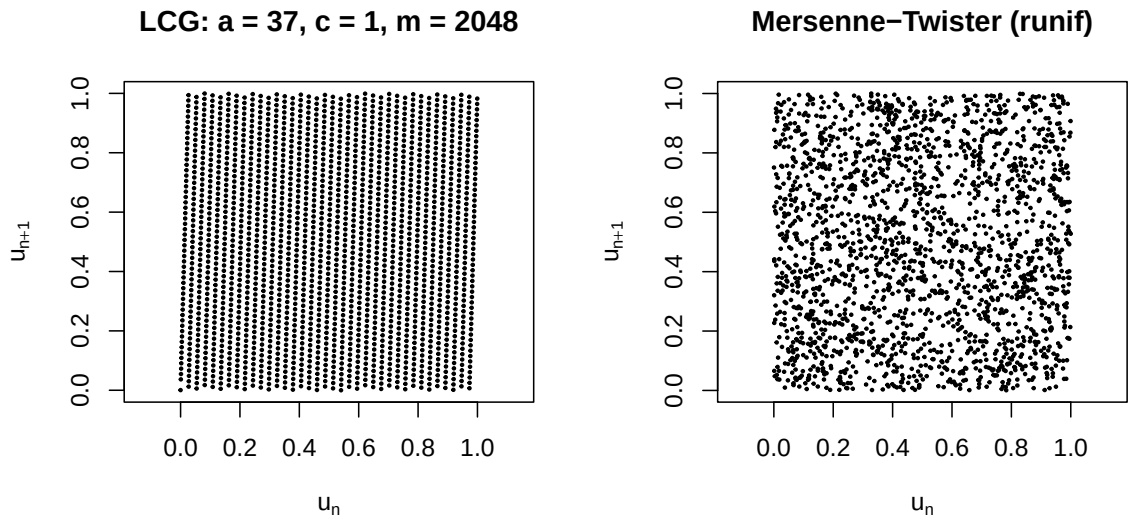
```
u_pm <- lcg(10000, seed = 42, a = 16807, c = 0, m = 2^31 - 1) / (2^31 - 1)
head(u_pm)
#> [1] 0.0003287075 0.5245871020 0.7354235322 0.2633055408 0.3762239713
#> [6] 0.1962858258
```

Die Gitterstruktur von LCGs

Ein LCG liefert gleichverteilte Einzelwerte. Betrachtet man aber **aufeinanderfolgende Paare** (u_n, u_{n+1}), liegen alle Punkte auf wenigen parallelen Geraden; in drei Dimensionen liegen Tripel auf wenigen Ebenen (Marsaglia 1968). Bei einem kleinen Modul sieht man das sofort — hier liegen alle 2048 Paare auf 37 Geraden mit Steigung $a = 37$:

```
u_klein <- lcg(2048, seed = 1, a = 37, c = 1, m = 2048) / 2048
set.seed(1)
u_mt <- runif(2048)

par(mfrow = c(1, 2))
plot(head(u_klein, -1), u_klein[-1], pch = 20, cex = 0.4, asp = 1,
     main = "LCG: a = 37, c = 1, m = 2048",
     xlab = expression(u[n]), ylab = expression(u[n + 1]))
plot(head(u_mt, -1), u_mt[-1], pch = 20, cex = 0.4, asp = 1,
     main = "Mersenne-Twister (runif)",
     xlab = expression(u[n]), ylab = expression(u[n + 1]))
```




```
par(mfrow = c(1, 1))
```

RANDU: ein berüchtigter Generator

RANDU war in den 1960er- und 1970er-Jahren auf IBM-Rechnern weit verbreitet. Wegen $a = 65539 = 2^{16} + 3$ gilt $a^2 \equiv 6a - 9 \pmod{2^{31}}$ und damit für **jedes** Tripel

$$x_{n+2} = 6\,x_{n+1} - 9\,x_n \pmod{2^{31}}.$$

Folglich ist $9u_n - 6u_{n+1} + u_{n+2}$ immer eine **ganze Zahl** — alle Tripel liegen auf nur 15 parallelen Ebenen im Einheitswürfel.

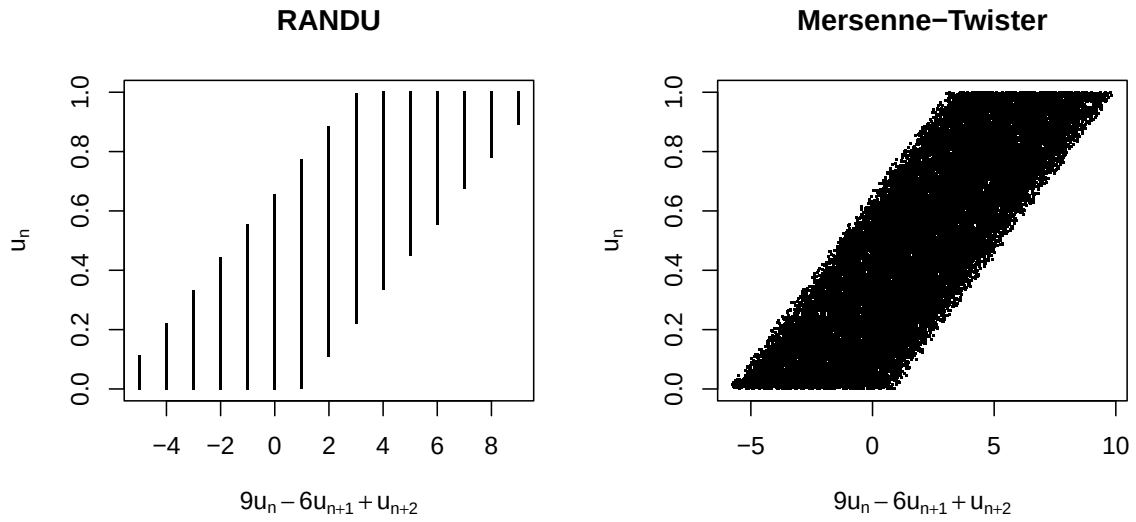
```
u_randu <- lcg(30000, seed = 7, a = 65539, c = 0, m = 2^31) / 2^31   # RANDU braucht einen u
n <- length(u_randu)
v <- 9 * u_randu[1:(n - 2)] - 6 * u_randu[2:(n - 1)] + u_randu[3:n]

all(v == round(v))      # wirklich immer ganzzahlig?
#> [1] TRUE
table(v)                # nur 15 verschiedene Werte
#> v
#>   -5   -4   -3   -2   -1    0    1    2    3    4    5    6    7    8    9
#>  287  832 1429 1991 2509 2893 3339 3275 3349 3134 2488 1990 1361  810  311
```

Schaut man „von der Seite” auf die Ebenen, werden sie sichtbar:

```r
set.seed(3)
u3 <- runif(30000)
w  <- 9 * u3[1:(n - 2)] - 6 * u3[2:(n - 1)] + u3[3:n]

par(mfrow = c(1, 2))
plot(v, u_randu[1:(n - 2)], pch = ".", main = "RANDU",
     xlab = expression(9*u[n] - 6*u[n+1] + u[n+2]), ylab = expression(u[n]))
plot(w, u3[1:(n - 2)], pch = ".", main = "Mersenne-Twister",
     xlab = expression(9*u[n] - 6*u[n+1] + u[n+2]), ylab = expression(u[n]))
```




```r
par(mfrow = c(1, 1))
```

1.3 Zufallszahlengeneratoren testen

Was wird eigentlich getestet?

Ein Generator ist nie „bewiesen gut" — er hat lediglich Tests bestanden. Jeder Test prüft dieselbe Nullhypothese

$$H_0: \; u_1, u_2, \ldots, u_n \text{ sind unabhängig und } U(0,1)\text{-verteilt},$$

aber jeder schaut auf einen anderen Aspekt. Man unterscheidet zwei Gruppen:

Eigenschaft	Prüfung	R
Gleichverteilung	Histogramm	`hist(u)`
	χ^2-Anpassungstest	`chisq.test(table(cut(u, k)))`
	Empirische Verteilungsfunktion, KS-Test	`plot(ecdf(u))`, `ks.test(u, "punif")`
Unabhängigkeit	Paar-Streudiagramm (Lag-Plot)	`plot(u[-n], u[-1])`
	Autokorrelation	`acf(u)`
	Runs-Test (Läufe)	eigene Funktion (unten)
	Serieller Test (Paare, Tripel)	eigene Funktion (unten)

Wir testen fünf „Generatoren": den Mersenne-Twister, Park & Miller, RANDU, die Mittel-Quadrat-Methode und eine absichtlich **zu regelmäßige** Folge, die einfach das Intervall $[0, 1)$ in gleichen Schritten abläuft.

```
set.seed(10)
gen <- list(
  Mersenne_Twister = runif(10000),
  Park_Miller      = u_pm,
  RANDU            = u_randu[1:10000],
  Mittel_Quadrat   = ms_folge(1234, 1000) / 10^4,
  Zu_regelmaessig  = ((1:10000) - 0.5) / 10000
)
sapply(gen, length)
#> Mersenne_Twister      Park_Miller            RANDU   Mittel_Quadrat
#>            10000            10000            10000             1000
#>  Zu_regelmaessig
#>            10000
```

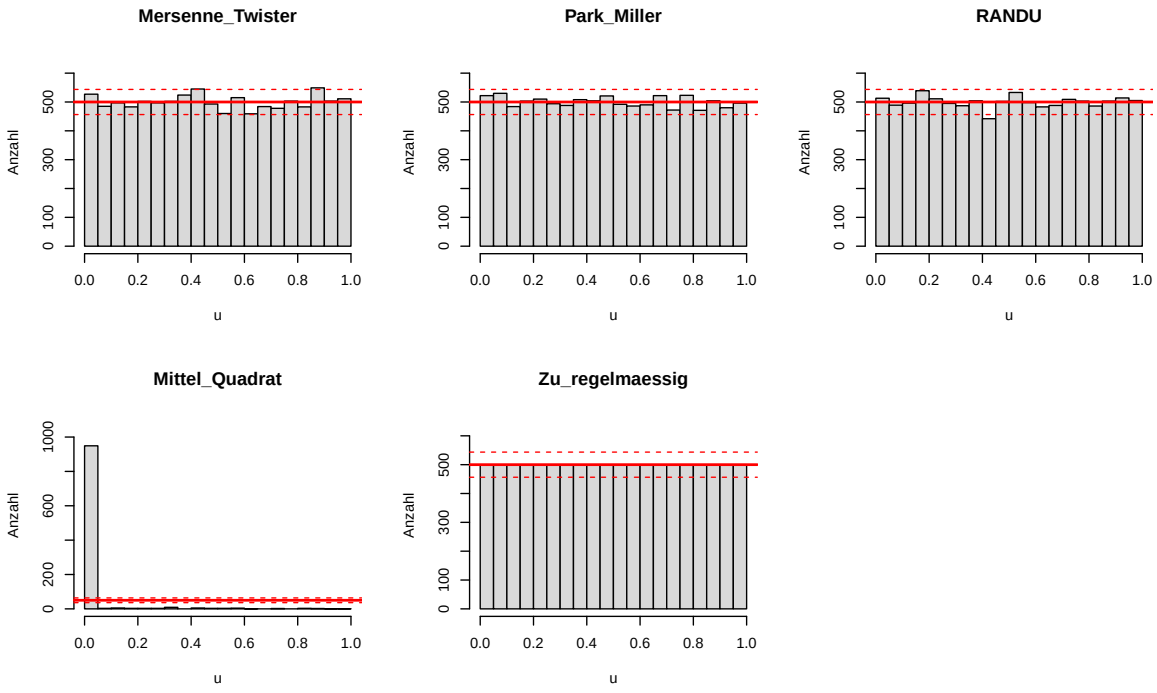
Histogramm

Bei n Werten und k gleich breiten Klassen ist die Anzahl in einer Klasse $\mathrm{Bin}(n, 1/k)$-verteilt: Erwartungswert n/k, Standardabweichung $\sqrt{n \cdot \frac{1}{k}(1 - \frac{1}{k})}$. Ein Histogramm ist **nie** perfekt flach — die Frage ist, ob die Schwankungen im erwarteten Rahmen bleiben. Die gestrichelten Linien zeigen ± 2 Standardabweichungen:

```
hist_band <- function(u, titel, k = 20) {
  e <- length(u) / k                                  # erwartete Anzahl je Klasse
  s <- sqrt(length(u) * (1 / k) * (1 - 1 / k))        # Standardabweichung
  h <- hist(u, breaks = seq(0, 1, length.out = k + 1), plot = FALSE)
  plot(h, main = titel, xlab = "u", ylab = "Anzahl", col = "grey85",
```

```
       ylim = c(0, 1.15 * max(h$counts, e + 3 * s)))
  abline(h = e, col = "red", lwd = 2)
  abline(h = e + c(-2, 2) * s, col = "red", lty = 2)
}
```

```
par(mfrow = c(2, 3))
for (nm in names(gen)) hist_band(gen[[nm]], nm)
par(mfrow = c(1, 1))
```



Die Mittel-Quadrat-Methode fällt sofort auf: Nach dem Vorlauf hängt sie auf einem einzigen Wert fest. RANDU und die regelmäßige Folge sehen dagegen tadellos aus — die regelmäßige Folge sogar *zu* gut: Alle Säulen sind exakt gleich hoch.

1.4 Die Monte-Carlo-Methode

Idee und Geschichte

Die Monte-Carlo-Methode wurde in den 1940er-Jahren von Stanisław Ulam, John von Neumann und Nicholas Metropolis in Los Alamos entwickelt, um die Ausbreitung von Neutronen zu berechnen. Der Name spielt auf das Spielcasino von Monte Carlo an.

Das Prinzip: **Eine unbekannte Größe wird als Erwartungswert geschrieben und durch einen Stichprobenmittelwert geschätzt.**

$$\theta = E[h(X)] \quad \Longrightarrow \quad \hat{\theta}_n = \frac{1}{n}\sum_{i=1}^{n} h(X_i), \qquad X_i \text{ unabhängig gezogen.}$$

Zwei Sätze garantieren, dass das funktioniert:

- **Gesetz der großen Zahlen:** $\hat{\theta}_n \to \theta$ für $n \to \infty$.
- **Zentraler Grenzwertsatz:** $\hat{\theta}_n$ ist näherungsweise normalverteilt mit Standardfehler $\mathrm{SE} = \sigma/\sqrt{n}$, wobei $\sigma^2 = \mathrm{Var}(h(X))$.

Daraus folgt die wichtigste Faustregel der Simulation:

> **!** Der Monte-Carlo-Fehler fällt mit $1/\sqrt{n}$
>
> Für **eine** zusätzliche korrekte Dezimalstelle (Fehler durch 10) braucht man **100-mal** so viele Wiederholungen. Dafür hängt die Rate nicht von der Dimension des Problems ab — der Grund, warum Monte Carlo in hohen Dimensionen konkurrenzlos ist.

Rezept für jede Simulation

1. Die gesuchte Größe als Wahrscheinlichkeit oder Erwartungswert formulieren.
2. Eine Funktion schreiben, die **ein** Zufallsexperiment durchführt.
3. Mit `set.seed()` den Zufallsstrom festlegen.
4. Das Experiment n-mal wiederholen (`replicate()` oder vektorisiert).
5. Mittelwert **und** Standardfehler berichten.

1.5 Monte-Carlo-Schätzung von π

Methode 1: Kreis im Quadrat

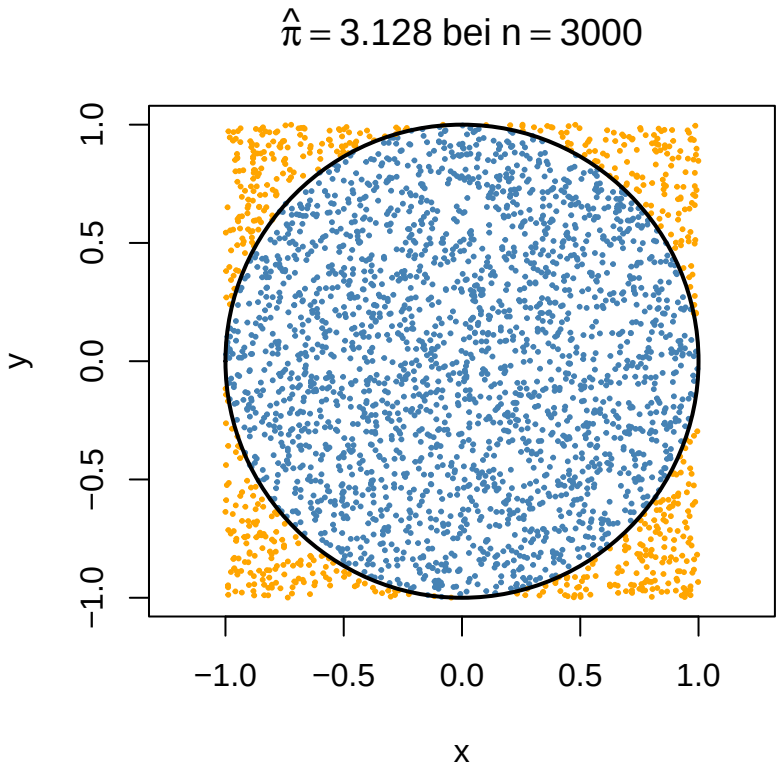
Wir ziehen Punkte (X, Y) gleichverteilt im Quadrat $[-1, 1]^2$ (Fläche 4). Der Einheitskreis hat Fläche π. Also gilt

$$p = P(X^2 + Y^2 \le 1) = \frac{\pi}{4} \quad \Longrightarrow \quad \hat{\pi} = 4 \cdot \frac{\text{Anzahl Punkte im Kreis}}{n}.$$

```r
set.seed(2026)
n <- 10000
x <- runif(n, -1, 1)
y <- runif(n, -1, 1)
im_kreis <- x^2 + y^2 <= 1
```

```r
pi_hat <- 4 * mean(im_kreis)
pi_hat
#> [1] 3.142
pi_hat - pi        # tatsächlicher Fehler
#> [1] 0.0004073464
```

```r
k <- 3000          # nur einen Teil zeichnen
plot(x[1:k], y[1:k], pch = 20, cex = 0.4, asp = 1,
     col = ifelse(im_kreis[1:k], "steelblue", "orange"),
     xlab = "x", ylab = "y",
     main = bquote(hat(pi) == .(4 * mean(im_kreis[1:k])) ~ "bei" ~ n == .(k)))
symbols(0, 0, circles = 1, inches = FALSE, add = TRUE, lwd = 2)
```



Standardfehler und Konfidenzintervall

Die Trefferindikatoren sind Bernoulli-verteilt mit Var $= p(1-p)$. Daher

$$\mathrm{SE}(\hat{\pi}) = 4\sqrt{\frac{\hat{p}(1-\hat{p})}{n}}.$$

```r
p_hat <- mean(im_kreis)
se    <- 4 * sqrt(p_hat * (1 - p_hat) / n)
se
#> [1] 0.016419
pi_hat + c(-1, 1) * qnorm(0.975) * se    # 95-%-Konfidenzintervall
#> [1] 3.109819 3.174181
```

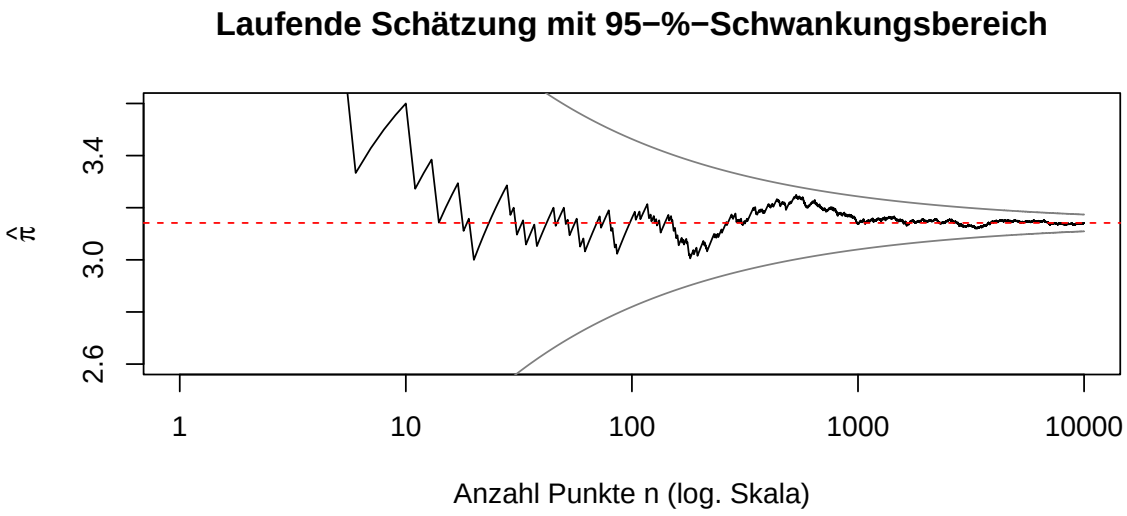
Ein Monte-Carlo-Ergebnis ohne Standardfehler ist unvollständig — genau wie ein Messwert ohne Messunsicherheit.

Konvergenz beobachten

`cumsum()` liefert die laufende Schätzung nach $1, 2, \ldots, n$ Punkten:

```r
laufend <- 4 * cumsum(im_kreis) / seq_len(n)
schritt <- seq_len(n)
band    <- qnorm(0.975) * 4 * sqrt((pi / 4) * (1 - pi / 4) / schritt)

plot(schritt, laufend, type = "l", log = "x", ylim = c(2.6, 3.6),
     xlab = "Anzahl Punkte n (log. Skala)", ylab = expression(hat(pi)),
     main = "Laufende Schätzung mit 95-%-Schwankungsbereich")
abline(h = pi, col = "red", lty = 2)
lines(schritt, pi + band, col = "grey50")
lines(schritt, pi - band, col = "grey50")
```



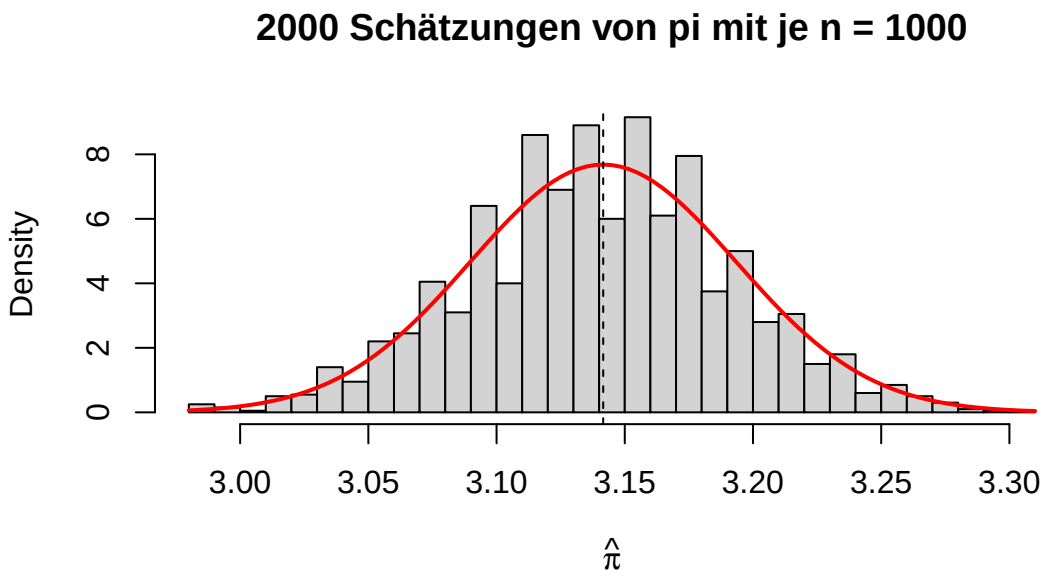
Die Schätzung ist selbst eine Zufallsvariable

Wiederholt man die ganze Simulation, erhält man jedes Mal ein anderes $\hat{\pi}$. Die Verteilung dieser Schätzungen ist die *Stichprobenverteilung* — und nach dem zentralen Grenzwertsatz annähernd normal:

```
pi_kreis <- function(n) {
  4 * mean(runif(n)^2 + runif(n)^2 <= 1)   # Viertelkreis genügt
}

set.seed(11)
schaetzungen <- replicate(2000, pi_kreis(1000))
mean(schaetzungen); sd(schaetzungen)
#> [1] 3.14124
#> [1] 0.05096981
4 * sqrt((pi / 4) * (1 - pi / 4) / 1000)     # theoretischer SE
#> [1] 0.0519304

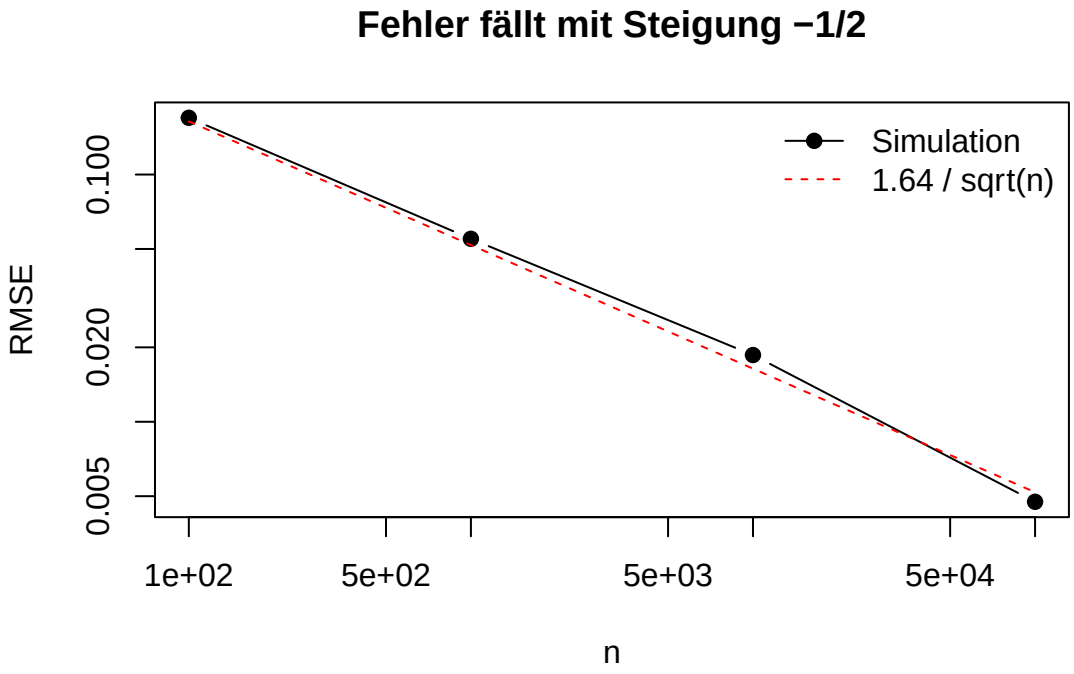
hist(schaetzungen, breaks = 40, freq = FALSE,
     main = "2000 Schätzungen von pi mit je n = 1000", xlab = expression(hat(pi)))
curve(dnorm(x, pi, 4 * sqrt((pi / 4) * (1 - pi / 4) / 1000)),
      add = TRUE, col = "red", lwd = 2)
abline(v = pi, lty = 2)
```



Die $1/\sqrt{n}$-Regel sichtbar machen

```
set.seed(12)
n_werte <- 10^(2:5)
rmse <- sapply(n_werte, function(n) {
  sqrt(mean((replicate(100, pi_kreis(n)) - pi)^2))
})
data.frame(n = n_werte, rmse = signif(rmse, 3))
#>       n    rmse
#> 1 1e+02 0.17000
#> 2 1e+03 0.05500
#> 3 1e+04 0.01860
#> 4 1e+05 0.00475

plot(n_werte, rmse, log = "xy", type = "b", pch = 19,
     xlab = "n", ylab = "RMSE", main = "Fehler fällt mit Steigung -1/2")
lines(n_werte, 1.64 / sqrt(n_werte), col = "red", lty = 2)
legend("topright", c("Simulation", "1.64 / sqrt(n)"),
       col = c("black", "red"), lty = c(1, 2), pch = c(19, NA), bty = "n")
```



Wie viele Punkte braucht man?

Soll der Fehler mit 95 % Sicherheit unter ε bleiben, muss $1{,}96 \cdot \sigma/\sqrt{n} \le \varepsilon$ gelten, also

$$n \ge \left(\frac{1{,}96\,\sigma}{\varepsilon}\right)^2, \qquad \sigma = 4\sqrt{p(1-p)} \approx 1{,}64.$$

```r
sigma <- 4 * sqrt(pi / 4 * (1 - pi / 4))
eps   <- 10^-(1:5)
data.frame(genauigkeit = eps,
           n_noetig = format(ceiling((1.96 * sigma / eps)^2), big.mark = ".",
                             decimal.mark = ",", scientific = FALSE))
#>   genauigkeit        n_noetig
#> 1       1e-01           1.036
#> 2       1e-02         103.599
#> 3       1e-03      10.359.898
#> 4       1e-04   1.035.989.709
#> 5       1e-05 103.598.970.849
```

Fünf korrekte Nachkommastellen kosten rund 100 Milliarden Punkte. Monte Carlo ist ein schlechtes Verfahren, um π zu berechnen — aber ein ideales Beispiel, weil wir die richtige Antwort kennen.

Methode 2: Buffons Nadel (1777)

Georges-Louis Leclerc, Comte de Buffon, stellte folgende Frage: Ein Boden besteht aus parallelen Dielen der Breite d. Eine Nadel der Länge $l \le d$ wird zufällig fallen gelassen. Wie groß ist die Wahrscheinlichkeit, dass sie eine Fuge kreuzt?

Beschreibe die Lage der Nadel durch

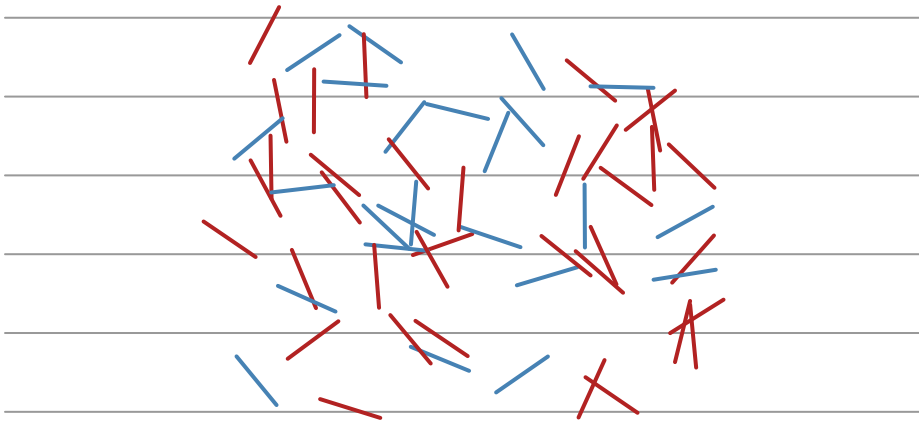
- Y = Abstand des Nadelmittelpunkts zur nächsten Fuge, $Y \sim U(0, d/2)$,
- Θ = spitzer Winkel zwischen Nadel und Fugen, $\Theta \sim U(0, \pi/2)$, unabhängig von Y.

Die Nadel kreuzt eine Fuge genau dann, wenn $Y \le \frac{l}{2}\sin\Theta$. Integration über beide Variablen ergibt

$$p = P\big(Y \le \tfrac{l}{2}\sin\Theta\big) = \frac{2}{\pi}\int_0^{\pi/2} \frac{l\,\sin\theta}{d}\,d\theta = \frac{2l}{\pi d} \quad\Longrightarrow\quad \hat{\pi} = \frac{2l}{d\,\hat{p}}.$$

```r
set.seed(7)
d <- 1; l <- 0.8
anz <- 60
mx <- runif(anz, 0, 6)                      # Mittelpunkte
my <- runif(anz, 0, 5)
w  <- runif(anz, 0, pi)                     # Richtung der Nadel
x1 <- mx - l / 2 * cos(w); x2 <- mx + l / 2 * cos(w)
y1 <- my - l / 2 * sin(w); y2 <- my + l / 2 * sin(w)
kreuzt <- floor(y1 / d) != floor(y2 / d)   # Endpunkte in verschiedenen Dielen

plot(NA, xlim = c(0, 6), ylim = c(-0.5, 5.5), asp = 1, axes = FALSE,
     xlab = "", ylab = "", main = sprintf("%d von %d Nadeln kreuzen eine Fuge",
                                          sum(kreuzt), anz))
abline(h = seq(-1, 6, by = d), col = "grey60")
segments(x1, y1, x2, y2, lwd = 2, col = ifelse(kreuzt, "firebrick", "steelblue"))
```




```
buffon <- function(n, l = 1, d = 1) {
  y     <- runif(n, 0, d / 2)
  theta <- runif(n, 0, pi / 2)
  p_hat <- mean(y <= (l / 2) * sin(theta))
  2 * l / (d * p_hat)
}

set.seed(2026)
buffon(10000)
#> [1] 3.156566
buffon(1e6)
#> [1] 3.140127
```

i Ein Zirkelschluss?

Um $\Theta \sim U(0, \pi/2)$ zu ziehen, benutzt `buffon()` bereits π. Echte Nadeln brauchen das nicht — die Simulation schon. Man kann den Winkel auch ohne π erzeugen: Einen Punkt

gleichverteilt im Viertelkreis ziehen (Verwerfungsmethode) und dessen Richtung nehmen.

```
buffon_ohne_pi <- function(n, l = 1, d = 1) {
  # Richtungen: Punkte im Einheitsquadrat ziehen, nur die im Viertelkreis behalten
  vx <- runif(2 * n); vy <- runif(2 * n)
  ok <- vx^2 + vy^2 <= 1 & vx^2 + vy^2 > 0
  vx <- vx[ok][1:n]; vy <- vy[ok][1:n]
  sin_theta <- vy / sqrt(vx^2 + vy^2)
  y <- runif(n, 0, d / 2)
  2 * l / (d * mean(y <= (l / 2) * sin_theta))
}

set.seed(2026)
buffon_ohne_pi(1e6)
#> [1] 3.142954
```

Welche Methode ist besser?

Beide Schätzer haben die Fehlerrate $1/\sqrt{n}$, aber unterschiedliche Konstanten. Mit der Delta-Methode erhält man für Buffon ($l = d$, $p = 2/\pi$)

$$\mathrm{SE}(\hat{\pi}_{\text{Buffon}}) \approx \frac{2}{p^2}\sqrt{\frac{p(1-p)}{n}} \approx \frac{2{,}37}{\sqrt{n}}, \qquad \mathrm{SE}(\hat{\pi}_{\text{Kreis}}) \approx \frac{1{,}64}{\sqrt{n}}.$$

```
set.seed(21)
c(kreis  = sd(replicate(2000, pi_kreis(1000))),
  buffon = sd(replicate(2000, buffon(1000))))
#>      kreis     buffon
#> 0.05160791 0.07383903
c(kreis_theorie  = 4 * sqrt(pi / 4 * (1 - pi / 4)) / sqrt(1000),
  buffon_theorie = 2 / (2 / pi)^2 * sqrt((2 / pi) * (1 - 2 / pi)) / sqrt(1000))
#>  kreis_theorie buffon_theorie
#>     0.05193040     0.07505687
```

Buffon braucht für dieselbe Genauigkeit etwa $(2{,}37/1{,}64)^2 \approx 2$-mal so viele Würfe.

⚠ Zu schön, um wahr zu sein: Lazzarini (1901)

Der Mathematiker Mario Lazzarini berichtete, er habe eine Nadel ($l/d = 5/6$) 3408-mal geworfen und 1808 Kreuzungen gezählt. Das ergibt $\hat{\pi} = 2 \cdot \frac{5}{6} \cdot 3408/1808 = 355/113 = 3{,}1415929$ — sechs korrekte Nachkommastellen. Nach unserer Tabelle bräuchte man dafür

mehr als 10^{13} Würfe. Die „krumme" Wurfzahl 3408 ist genau so gewählt, dass der bekannte Bruch 355/113 herauskommt.

```
p_laz <- 2 * (5 / 6) / pi                              # Kreuzungswahrscheinlichkeit
2 * (5 / 6) * 3408 / 1808                               # Lazzarinis Schätzung
#> [1] 3.141593
dbinom(1808, size = 3408, prob = p_laz)                 # P(genau 1808 Kreuzungen)
#> [1] 0.01369206
sd_laz <- 2 * (5/6) / p_laz^2 * sqrt(p_laz * (1 - p_laz) / 3408)
sd_laz                                                  # typischer Fehler bei n = 3408
#> [1] 0.05062451
```

Ein ehrlicher Versuch mit 3408 Würfen hat einen typischen Fehler von etwa 0,05 — Lazzarinis Ergebnis ($355/113 - \pi \approx 2{,}7 \cdot 10^{-7}$) liegt mehr als 100 000-mal näher an π. Lehre für die Praxis: Die Simulation selbst sagt Ihnen, welche Genauigkeit plausibel ist.

1.6 Monte-Carlo-Simulation: Kollisionen in Hashtabellen

Ausgangslage

Eine Hashtabelle speichert Schlüssel in m *Buckets* (Slots). Eine Hashfunktion h bildet jeden Schlüssel auf einen Index $h(k) \in \{1, \ldots, m\}$ ab. Landen zwei Schlüssel im selben Slot, liegt eine **Kollision** vor.

Für die Analyse verwenden wir das Modell des **einfachen uniformen Hashings**: Jeder Schlüssel landet unabhängig und gleichverteilt in einem der m Slots. In R ist das genau

```
set.seed(1)
m <- 10
sample.int(m, size = 6, replace = TRUE)    # Slots von 6 Schlüsseln
#> [1] 9 4 7 1 2 7
```

Frage 1: Wie wahrscheinlich ist mindestens eine Kollision?

Das ist das **Geburtstagsproblem**: n Personen, $m = 365$ Tage. Die Wahrscheinlichkeit, dass alle n Schlüssel in verschiedenen Slots landen, ist

$$P(\text{keine Kollision}) = \prod_{i=0}^{n-1} \left(1 - \frac{i}{m}\right) \approx \exp\left(-\frac{n(n-1)}{2m}\right).$$

```r
p_kollision_exakt  <- function(n, m) 1 - prod(1 - (0:(n - 1)) / m)
p_kollision_approx <- function(n, m) 1 - exp(-n * (n - 1) / (2 * m))

p_kollision_sim <- function(n, m, R = 10000) {
  mean(replicate(R, anyDuplicated(sample.int(m, n, replace = TRUE)) > 0))
}

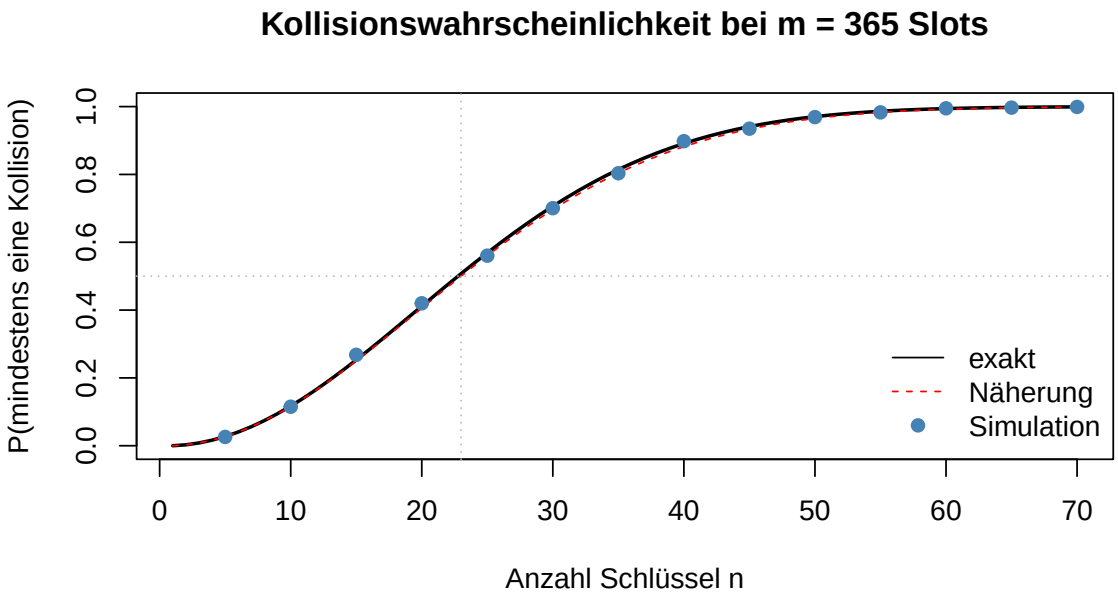
set.seed(365)
c(exakt  = p_kollision_exakt(23, 365),
  approx = p_kollision_approx(23, 365),
  sim    = p_kollision_sim(23, 365))
#>     exakt    approx       sim
#> 0.5072972 0.5000018 0.5068000
```

Schon bei 23 Schlüsseln in 365 Slots ist eine Kollision wahrscheinlicher als keine. `anyDuplicated()` liefert die Position des ersten doppelten Werts (0, falls keiner existiert) — schneller als `any(duplicated())`.

```r
m <- 365
n_seq <- 1:70
exakt <- sapply(n_seq, p_kollision_exakt, m = m)

set.seed(1)
n_sim <- seq(5, 70, by = 5)
sim   <- sapply(n_sim, p_kollision_sim, m = m, R = 2000)

plot(n_seq, exakt, type = "l", lwd = 2, ylim = c(0, 1),
     xlab = "Anzahl Schlüssel n", ylab = "P(mindestens eine Kollision)",
     main = "Kollisionswahrscheinlichkeit bei m = 365 Slots")
lines(n_seq, p_kollision_approx(n_seq, m), col = "red", lty = 2)
points(n_sim, sim, pch = 19, col = "steelblue")
abline(h = 0.5, v = 23, col = "grey70", lty = 3)
legend("bottomright", c("exakt", "Näherung", "Simulation"),
       col = c("black", "red", "steelblue"), lty = c(1, 2, NA),
       pch = c(NA, NA, 19), bty = "n")
```



Die Geburtstagsschranke

Setzt man die Näherung gleich $1/2$, erhält man $n_{50\%} \approx \sqrt{2 \ln 2 \cdot m} \approx 1{,}18\sqrt{m}$. Kollisionen treten also schon bei etwa $\sqrt{m}$ Schlüsseln auf — **weit** vor m.

```
bits <- c(16, 32, 64, 128, 160, 256)
data.frame(hash_bits = bits,
           slots     = sprintf("2^%d", bits),
           n_50      = signif(sqrt(2 * log(2) * 2^bits), 3),
           ca        = sprintf("2^%.0f", bits / 2))
#>   hash_bits slots     n_50    ca
#> 1        16  2^16 3.01e+02   2^8
#> 2        32  2^32 7.72e+04  2^16
#> 3        64  2^64 5.06e+09  2^32
#> 4       128 2^128 2.17e+19  2^64
#> 5       160 2^160 1.42e+24  2^80
#> 6       256 2^256 4.01e+38 2^128
```

i Bedeutung für die Kryptografie

Eine kryptografische Hashfunktion mit b Bit bietet gegen das Finden *irgendeiner* Kollision nur etwa $2^{b/2}$ Sicherheit (*Geburtstagsangriff*). Bei MD5 ($b = 128$) sind das 2^{64} Versuche; durch zusätzliche Schwächen ist MD5 heute praktisch gebrochen. Für SHA-1 ($b = 160$)

wurde 2017 die erste öffentliche Kollision vorgestellt. Deshalb verwendet man heute SHA-256 oder SHA-3.

Frage 2: Wie viele Kollisionen sind zu erwarten?

Bei n Schlüsseln und m Slots ist ein bestimmter Slot mit Wahrscheinlichkeit $(1-1/m)^n$ leer. Die erwartete Anzahl belegter Slots ist daher $m\,(1-(1-1/m)^n)$ und die erwartete Anzahl Schlüssel, die in einem bereits belegten Slot landen,

$$E[\text{Kollisionen}] = n - m\left(1-\left(1-\tfrac{1}{m}\right)^n\right).$$

```r
kollisionen_sim <- function(n, m) n - length(unique(sample.int(m, n, replace = TRUE)))
kollisionen_erw <- function(n, m) n - m * (1 - (1 - 1 / m)^n)

set.seed(3)
m <- 1000
lastfaktoren <- c(0.25, 0.5, 0.75, 1, 2)
data.frame(
  alpha      = lastfaktoren,
  n          = lastfaktoren * m,
  theorie    = round(kollisionen_erw(lastfaktoren * m, m), 1),
  simulation = sapply(lastfaktoren * m, function(n)
                 mean(replicate(1000, kollisionen_sim(n, m))))
)
#>   alpha    n theorie simulation
#> 1  0.25  250    28.7     28.679
#> 2  0.50  500   106.4    106.676
#> 3  0.75  750   222.2    222.323
#> 4  1.00 1000   367.7    367.855
#> 5  2.00 2000  1135.2   1135.217
```

Der **Belegungsfaktor** (*load factor*) $\alpha = n/m$ ist die zentrale Kenngröße: Bei $\alpha = 1$ landen bereits etwa 37 % der Schlüssel ($1/e$) in einem schon belegten Slot.

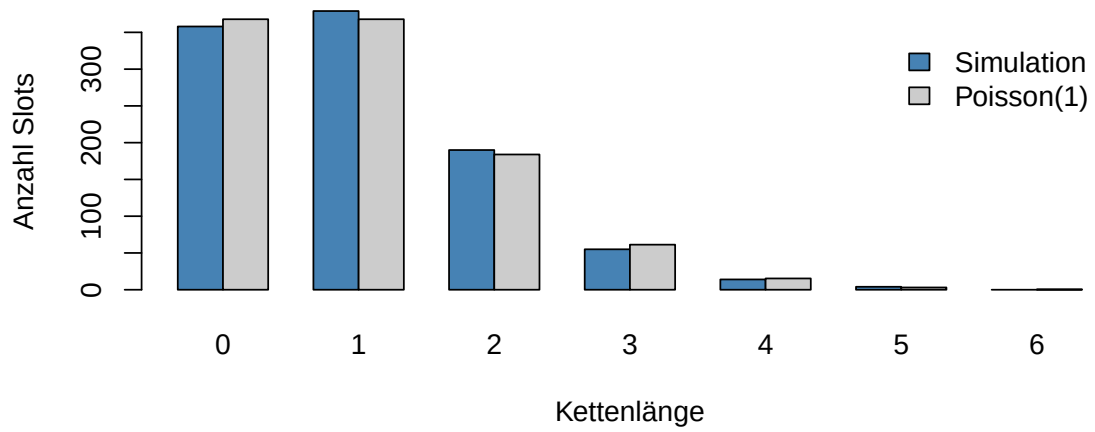
Frage 3: Wie lang werden die Ketten? (Hashing mit Verkettung)

Bei *Verkettung* (*separate chaining*) hängt in jedem Slot eine Liste der dort gelandeten Schlüssel. Die Kettenlänge eines Slots ist $\mathrm{Bin}(n, 1/m)$-verteilt, für großes m also näherungsweise $\mathrm{Poisson}(\alpha)$.

```r
set.seed(4)
m <- 1000; n <- 1000                          # alpha = 1
laengen <- tabulate(sample.int(m, n, replace = TRUE), nbins = m)

beobachtet <- table(factor(laengen, levels = 0:6))
erwartet   <- dpois(0:6, lambda = n / m) * m
rbind(beobachtet = as.vector(beobachtet), poisson = round(erwartet, 1))
#>             [,1]  [,2]  [,3] [,4] [,5] [,6] [,7]
#> beobachtet 358.0 379.0 190.0 55.0 14.0  4.0  0.0
#> poisson    367.9 367.9 183.9 61.3 15.3  3.1  0.5

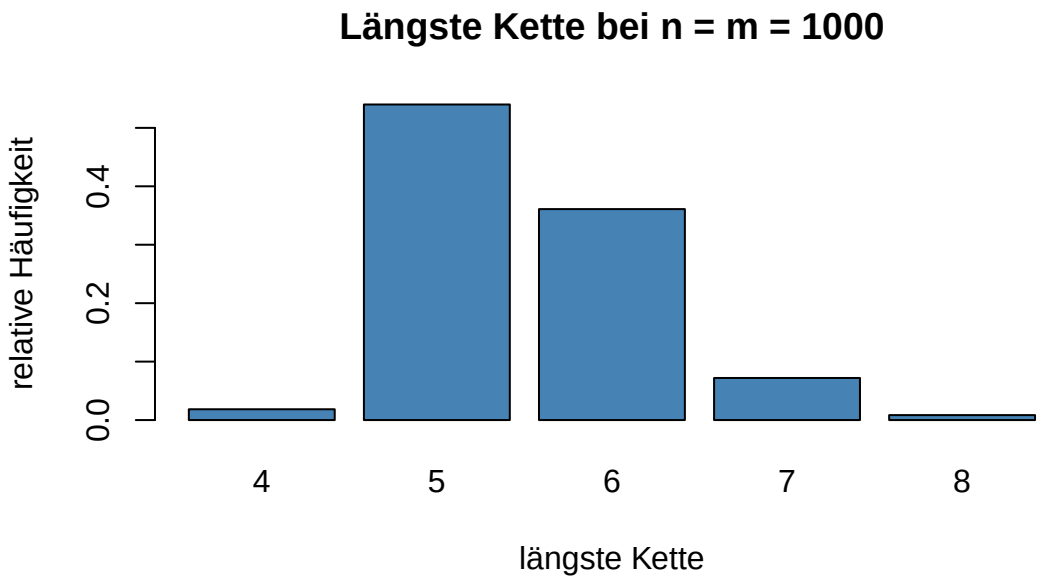
barplot(rbind(as.vector(beobachtet), erwartet), beside = TRUE,
        names.arg = 0:6, col = c("steelblue", "grey80"),
        xlab = "Kettenlänge", ylab = "Anzahl Slots",
        legend.text = c("Simulation", "Poisson(1)"),
        args.legend = list(bty = "n"))
```



Die mittlere Kettenlänge ist α — die Suchzeit im Mittel also $O(1+\alpha)$. Für die Worst-Case-Analyse interessiert aber die **längste** Kette. Ihre Verteilung ist analytisch schwierig, per Simulation trivial:

```r
set.seed(5)
max_kette <- replicate(2000, max(tabulate(sample.int(1000, 1000, replace = TRUE),
                                          nbins = 1000)))
table(max_kette)
```

```
#> max_kette
#>    4    5    6    7    8
#>   37 1080  722  144   17
barplot(table(max_kette) / 2000, col = "steelblue",
        xlab = "längste Kette", ylab = "relative Häufigkeit",
        main = "Längste Kette bei n = m = 1000")
```



Frage 4: Wie viele Sondierungen? (Offene Adressierung)

Bei *linearer Sondierung* (*linear probing*) wird ein Schlüssel, dessen Slot belegt ist, im nächsten freien Slot abgelegt. Dadurch bilden sich zusammenhängende Blöcke (*primäres Clustering*), und die Suche wird langsamer als bei idealem Hashing. Knuth (1963) hat für die erwartete Anzahl Sondierungen einer **erfolglosen Suche** hergeleitet:

$$E[\text{Sondierungen}] \approx \frac{1}{2}\left(1 + \frac{1}{(1-\alpha)^2}\right) \qquad \text{(lineare Sondierung)}$$

im Vergleich zu $1/(1-\alpha)$ bei idealem (uniformem) Sondieren. Wir prüfen das per Simulation:

```
lin_fuellen <- function(m, n) {
  belegt <- logical(m)
  for (k in seq_len(n)) {
    h <- sample.int(m, 1)
```

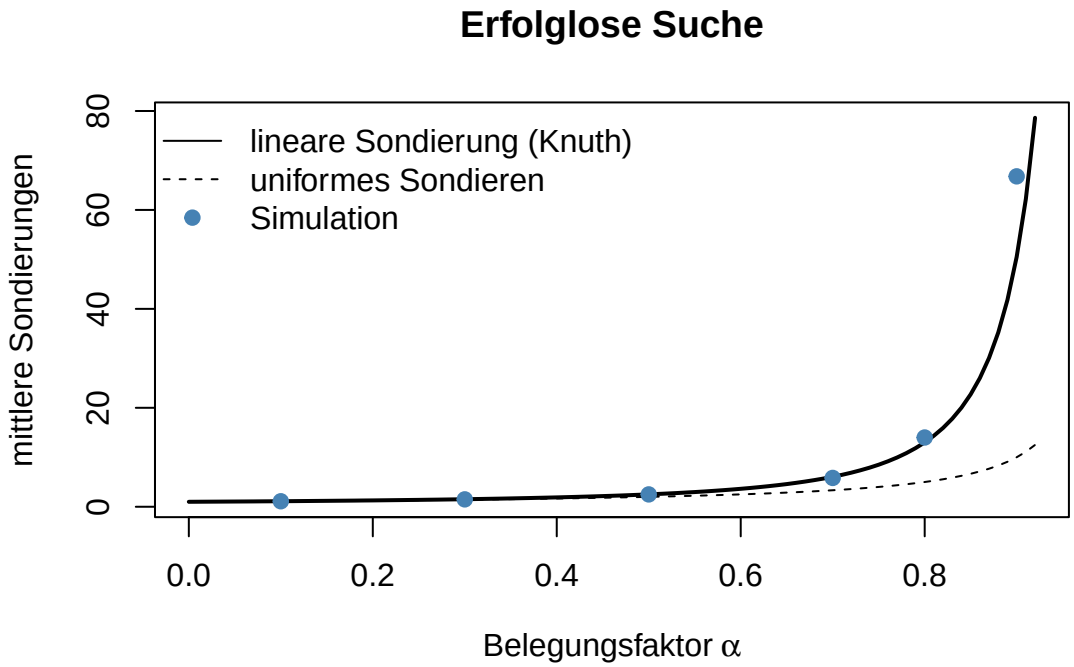
```
    while (belegt[h]) h <- h %% m + 1      # nächster Slot, am Ende wieder vorne
    belegt[h] <- TRUE
  }
  belegt
}

sondierungen <- function(belegt, R = 2000) {
  m <- length(belegt)
  sapply(sample.int(m, R, replace = TRUE), function(h) {
    s <- 1
    while (belegt[h]) { h <- h %% m + 1; s <- s + 1 }
    s
  })
}

set.seed(6)
alphas <- c(0.1, 0.3, 0.5, 0.7, 0.8, 0.9)
m <- 2000
sim_sond <- sapply(alphas, function(a) mean(sondierungen(lin_fuellen(m, a * m))))

data.frame(alpha      = alphas,
           simulation = round(sim_sond, 2),
           knuth      = round(0.5 * (1 + 1 / (1 - alphas)^2), 2),
           uniform    = round(1 / (1 - alphas), 2))
#>   alpha simulation knuth uniform
#> 1   0.1       1.11  1.12    1.11
#> 2   0.3       1.49  1.52    1.43
#> 3   0.5       2.49  2.50    2.00
#> 4   0.7       5.83  6.06    3.33
#> 5   0.8      14.00 13.00    5.00
#> 6   0.9      66.79 50.50   10.00
```

```
a_seq <- seq(0, 0.92, by = 0.01)
plot(a_seq, 0.5 * (1 + 1 / (1 - a_seq)^2), type = "l", lwd = 2,
     xlab = expression("Belegungsfaktor" ~ alpha),
     ylab = "mittlere Sondierungen", main = "Erfolglose Suche")
lines(a_seq, 1 / (1 - a_seq), lty = 2)
points(alphas, sim_sond, pch = 19, col = "steelblue")
legend("topleft", c("lineare Sondierung (Knuth)", "uniformes Sondieren",
                    "Simulation"),
       lty = c(1, 2, NA), pch = c(NA, NA, 19),
       col = c("black", "black", "steelblue"), bty = "n")
```



Bei kleinem α stimmen Simulation und Formel gut überein. Bei $\alpha \geq 0{,}8$ weicht die Simulation stärker ab: Knuths Formel gilt für $m \to \infty$, und die Sondierungszahl hängt dann stark davon ab, ob die Suche zufällig in einem der wenigen sehr langen Cluster startet — die Streuung ist groß. Praktische Konsequenz: Tabellen mit linearer Sondierung vergrößert man, bevor α etwa 0,7 erreicht.

Frage 5: Hält eine echte Hashfunktion, was das Modell verspricht?

Bisher haben wir **angenommen**, dass h uniform streut. Ob eine konkrete Hashfunktion das tut, kann man mit genau denselben Werkzeugen prüfen. Wir hashen 5000 typische Schlüssel `"user1"`, …, `"user5000"` in $m = 1009$ Slots (eine Primzahl) mit zwei Funktionen:

- **schlecht:** Summe der Zeichencodes modulo m,
- **besser:** Polynom-Hash $h = (\ldots((c_1 \cdot 31 + c_2) \cdot 31 + c_3)\ldots) \bmod m$, wie er z. B. in Java für Zeichenketten verwendet wird.

```
hash_summe <- function(s, m) sum(utf8ToInt(s)) %% m + 1

hash_polynom <- function(s, m, basis = 31) {
  h <- 0
  for (ch in utf8ToInt(s)) h <- (h * basis + ch) %% m
  h + 1
```

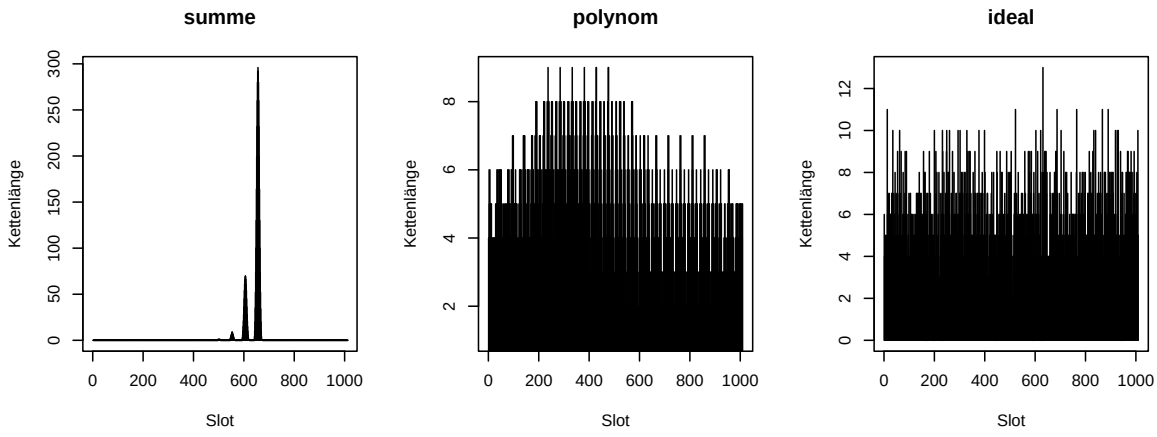
```
}

schluessel <- paste0("user", 1:5000)
m <- 1009

slots_summe   <- sapply(schluessel, hash_summe, m = m)
slots_polynom <- sapply(schluessel, hash_polynom, m = m)
set.seed(8)
slots_ideal   <- sample.int(m, length(schluessel), replace = TRUE)

kette <- function(slots) tabulate(slots, nbins = m)
round(sapply(list(summe = slots_summe, polynom = slots_polynom, ideal = slots_ideal),
       function(s) c(belegte_slots = sum(kette(s) > 0),
                     laengste_kette = max(kette(s)),
                     chisq_p = chisq.test(kette(s))$p.value)), 4)
#>                summe polynom     ideal
#> belegte_slots     85    1009 1001.0000
#> laengste_kette   296       9   13.0000
#> chisq_p            0       1    0.7561
```

```
par(mfrow = c(1, 3))
for (nm in c("summe", "polynom", "ideal")) {
  s <- get(paste0("slots_", nm))
  plot(kette(s), type = "h", main = nm,       # Achsen beachten!
       xlab = "Slot", ylab = "Kettenlänge")
}
```



```r
par(mfrow = c(1, 1))
```

Die Summen-Hashfunktion ignoriert die Reihenfolge der Zeichen (`"user12"` und `"user21"` kollidieren immer), und die Summen ähnlicher Schlüssel liegen dicht beieinander — alle 5000 Schlüssel landen in weniger als 100 Slots, die längste Kette hat fast 300 Einträge. Der Polynom-Hash belegt alle Slots und streut sogar *gleichmäßiger* als der Zufall ($p \approx 1$ im χ^2-Test): Aufeinanderfolgende Schlüssel unterscheiden sich nur in der letzten Ziffer und werden auf benachbarte Slots verteilt. Für solche regelmäßigen Schlüssel ist das ein Vorteil; bei bösartig gewählten Schlüsseln (*Hash-Flooding*) kann man jeden festen Polynom-Hash aber gezielt zum Kollidieren bringen. Monte Carlo liefert hier den **Referenzwert**, gegen den man eine reale Implementierung testet.

1.7 Ergänzungen

Monte-Carlo-Integration

Jedes Integral über $[a, b]$ ist ein Erwartungswert:

$$\int_a^b g(x)\,dx = (b-a)\,E[g(U)], \qquad U \sim U(a,b).$$

Beispiel: $\int_0^1 e^{-x^2}\,dx$ hat keine elementare Stammfunktion.

```r
g <- function(x) exp(-x^2)

set.seed(9)
n  <- 10000
gu <- g(runif(n))
c(schaetzung = mean(gu), se = sd(gu) / sqrt(n))
#>  schaetzung          se
#> 0.744170895 0.002010488
integrate(g, 0, 1)$value        # numerische Kontrolle
#> [1] 0.7468241
```

Die π-Schätzung mit dem Kreis war ein Spezialfall: Dort war g eine Indikatorfunktion (*Hit-or-Miss*-Methode). Die *Mittelwert-Methode* oben ist meist genauer, weil g weniger streut als ein 0/1-Indikator.

Hohe Dimensionen: Volumen der Einheitskugel

Das Volumen der d-dimensionalen Einheitskugel ist $V_d = \pi^{d/2}/\Gamma(d/2+1)$. Per Monte Carlo: Punkte im Würfel $[-1,1]^d$ (Volumen 2^d) ziehen und den Anteil in der Kugel zählen.

```r
kugel_mc <- function(d, n = 1e5) {
  X <- matrix(runif(n * d, -1, 1), ncol = d)
  anteil <- mean(rowSums(X^2) <= 1)
  c(schaetzung = 2^d * anteil, anteil = anteil)
}

set.seed(10)
dims <- c(2, 3, 5, 8, 10)
t(sapply(dims, function(d)
  c(d = d, kugel_mc(d), exakt = pi^(d / 2) / gamma(d / 2 + 1))))
#>       d schaetzung  anteil    exakt
#> [1,]  2    3.14252 0.78563 3.141593
#> [2,]  3    4.18360 0.52295 4.188790
#> [3,]  5    5.25728 0.16429 5.263789
#> [4,]  8    4.11136 0.01606 4.058712
#> [5,] 10    2.36544 0.00231 2.550164
```

Der Anteil der Punkte in der Kugel schrumpft mit wachsender Dimension dramatisch — in hohen Dimensionen liegt fast das ganze Volumen eines Würfels „in den Ecken". Hit-or-Miss wird dann ineffizient; trotzdem fällt der Fehler weiterhin mit $1/\sqrt{n}$, während Gitterverfahren mit $n = k^d$ Punkten exponentiell teurer werden.

Varianzreduktion: antithetische Variablen

Ist g monoton, sind $g(U)$ und $g(1-U)$ negativ korreliert. Ihr Mittelwert streut deshalb weniger als zwei unabhängige Werte — gleicher Rechenaufwand, kleinerer Fehler:

```r
set.seed(9)
u <- runif(n / 2)
anti <- (g(u) + g(1 - u)) / 2              # n/2 Paare = n Funktionsauswertungen
c(schaetzung = mean(anti),
  se_antithetisch = sd(anti) / sqrt(n / 2),
  se_einfach      = sd(gu) / sqrt(n))
#>      schaetzung se_antithetisch      se_einfach
#>    0.7468925399    0.0003993143    0.0020104880
```

Das Ziegenproblem (Monty Hall)

Hinter einer von drei Türen steht ein Auto. Sie wählen eine Tür; der Moderator öffnet eine *andere* Tür mit einer Ziege und bietet einen Wechsel an. Lohnt sich der Wechsel? Die Intuition irrt hier erfahrungsgemäß — die Simulation nicht:

```r
monty_hall <- function(wechseln) {
  auto  <- sample(1:3, 1)
  wahl  <- sample(1:3, 1)
  moeglich <- setdiff(1:3, c(auto, wahl))
  offen <- if (length(moeglich) == 1) moeglich else sample(moeglich, 1)
  if (wechseln) wahl <- setdiff(1:3, c(wahl, offen))
  wahl == auto
}

set.seed(3)
c(bleiben  = mean(replicate(10000, monty_hall(FALSE))),
  wechseln = mean(replicate(10000, monty_hall(TRUE))))
#>  bleiben wechseln
#>   0.3464   0.6524
```

> ⚠ **`sample()` mit einem einzelnen Wert**
>
> `sample(5, 1)` zieht aus `1:5`, nicht den Wert 5! Deshalb fragt `monty_hall()` die Länge von `moeglich` ab, bevor gezogen wird. Diese Falle steckt in vielen Simulationen.

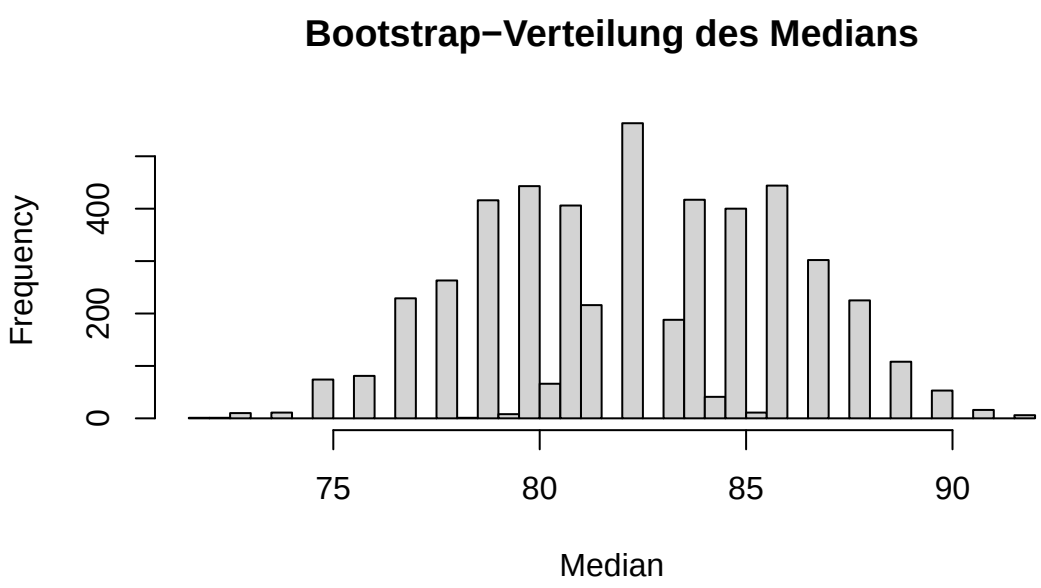
Ausblick: Bootstrap — Monte Carlo mit der eigenen Stichprobe

Bisher kannten wir die Verteilung, aus der gezogen wird. In der Praxis haben wir nur eine Stichprobe. Der **Bootstrap** (Efron 1979) zieht deshalb mit Zurücklegen aus der Stichprobe selbst und behandelt sie als Ersatz für die Population. So lässt sich der Standardfehler fast jeder Statistik schätzen — auch des Medians, für den es keine einfache Formel gibt.

```r
punkte <- c(88, 92, 75, 84, 79, 95, 68, 81, 90, 73, 86, 77)

set.seed(42)
boot_median <- replicate(5000, median(sample(punkte, replace = TRUE)))

median(punkte)
#> [1] 82.5
sd(boot_median)                                  # Bootstrap-Standardfehler
#> [1] 3.538501
quantile(boot_median, c(0.025, 0.975))           # Perzentil-Konfidenzintervall
#>  2.5% 97.5%
#>    76    89
hist(boot_median, breaks = 30, main = "Bootstrap-Verteilung des Medians",
     xlab = "Median")
```


Bootstrap−Verteilung des Medians
Frequency
0
200
400
75
80
85
90
Median